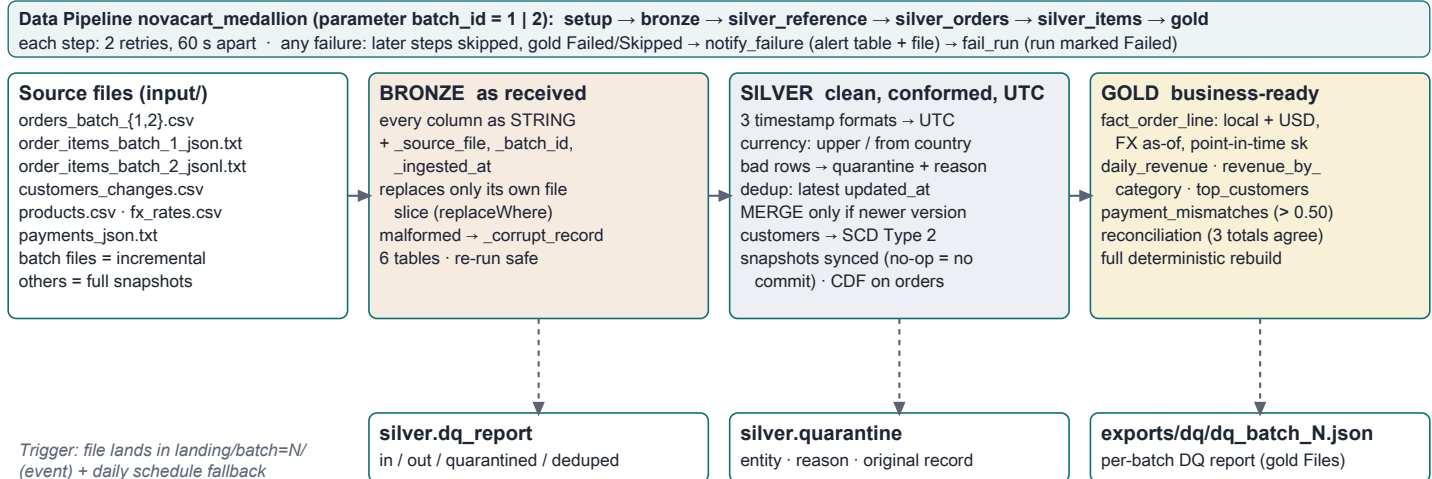


1 Tools chosen and why

OneLake lakehouses Bronze / Silver / Gold; raw input/ files kept, never edited	Delta Lake MERGE · HISTORY · time travel · NOT NULL + CHECK · Change Data Feed	Fabric Spark PySpark + Spark SQL notebooks; scales out, tiny F2 for the lab	Data Pipelines batch_id parameter · 2 retries · on-failure path · run monitoring	Entra ID + roles no keys or secrets anywhere; workspace roles for access
--	--	---	--	--

One platform covers every requirement natively, and Delta keeps the Spark code close to a Databricks design, so every line stays explainable. **Considered:** Azure Databricks + ADLS Gen2 (our first plan; the student subscription was disabled and the trial account could not grant storage roles), Databricks Free Edition, AWS S3 + Glue + Step Functions. Cost: F2 capacity (~\$0.36/h) resumed only while running, paused after.

2 Working pipeline architecture



3 Correct after batch 1, after batch 2, and on a re-run

Newer wins: NC-100473 is *paid* (updated 15 Sep) in batch 1 and *delivered* (19 Sep) in batch 2 → updated. **Late copy ignored:** NC-100276 is *cancelled* (10 Sep) in batch 1; batch 2 carries an older *paid* copy (9 Sep) → ignored, so it never re-enters revenue (10 such copies in the data). **silver.orders history:** v0–v2 create + constraints, **v3 = batch 1** (495 inserted), **v4 = batch 2** (450 inserted, 137 updated); **batch 2 again: 0 changes and no new version.** SELECT * FROM silver.orders VERSION AS OF 3 returns the state right after batch 1. Bronze replaces its own file slice, silver's guard is false for equal-or-older versions, quarantine and dq_report replace their own (entity, batch) rows, gold is rebuilt deterministically. Gold after batch 1 (\$150,062.64) and after batch 2 (\$295,585.57) equal an independent plain-Python calculation to the cent.

4 Assumptions where the spec was unclear

Timestamps	No offset ⇒ UTC; dd/MM/yyyy parsed explicitly; order_date = UTC date of order_ts	FX	Order-date rate, else latest earlier rate; USD = 1; none ⇒ fx_missing (excluded, counted)
Same updated_at	Later batch wins only if the content differs, so batch 2 then 1 = 1 then 2	Item key	(order_id, line_no); a later batch replaces a line only if it differs
Returns	qty forced to - qty ; dated by the parent order	Discount	NULL = 0; non-numeric or outside 0–100 ⇒ quarantine
Bad values	Unknown status or currency ⇒ quarantine; UK = GB; codes case-insensitive	Orphan items	Checked against all orders loaded so far ⇒ ORDER_NOT_FOUND
SCD2	Tier/country Type 2; name/e-mail Type 1; first version from 1900-01-01; key = hash(id, valid_from)	Payments	success +, refunded –, failed ignored; USD at the order-date rate
Mismatch	Expected = order total if paid/shipped/delivered, else 0; flagged if difference > 0.50	Unknown-order payments	Kept in silver, excluded from mismatches, counted (5)
Snapshots	Customers, products, FX, payments are full files reloaded each batch; payments cover the whole month	Top 10	Excludes the unknown customer (-1); ties broken by customer_id

5 Trigger design

File-arrival event (storage-event trigger on landing/batch=N/, batch_id from the folder) because batch 2's arrival time is unknown: it runs exactly when data lands and never idles. A **fixed schedule** would run too early or too late; a **windowed schedule** suits regular time slices with backfill but still polls. A daily schedule stays as a safety net, which is harmless because every step is idempotent.

Next with more time: auto-retry of quarantined orphan lines · Change Data Feed on order_items and incremental gold in production (tested: 0 differences from the full rebuild) · Activator event trigger and an Outlook/Teams alert · Fabric Git integration + deployment pipelines (dev/test/prod) with CI tests · DQ expectations as code · Power BI semantic model and a DQ dashboard on gold.